

SIH PS 26073 Engineering Blueprint: Self-Aware & Self-Healing AWS Network

Project Goal: Build a production-grade, 3-tier hybrid quality control (QC) and predictive maintenance platform for Automatic Weather Stations (AWS). The system ingests T (Temperature), P (Pressure), and H (Humidity), differentiates genuine severe weather from hardware failures, predicts sensor degradation, and estimates corrected values.

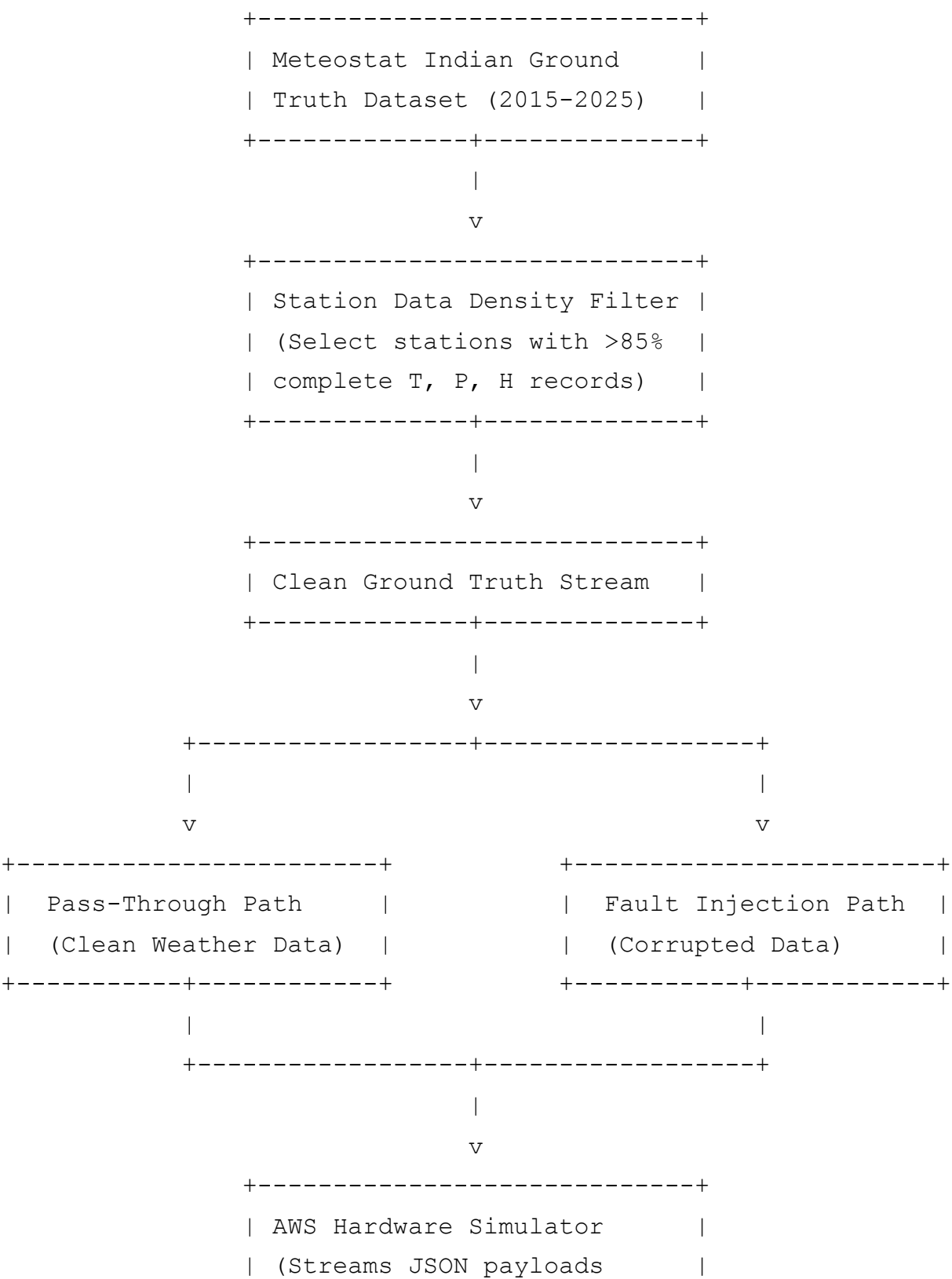
1. Critical Technical Gaps & Solutions

Identified Risk / Gap	Why It Breaks the System	How We Fix It in This Plan
1. Sparse Meteostat Data	Filtering all Indian stations blindly leads to stations with missing P or H blocks, breaking multi-variate ML.	Filter stations programmatically using a strict 85%+ completeness threshold across 2015–2025. Lock down 3–5 spatial clusters (e.g., Delhi, Mumbai, Pune, Chennai).
2. SHAP API Latency (>2s)	Computing dynamic SHAP values per incoming payload destroys real-time API performance (<50ms requirement).	Use Reconstruction Error Vectors for sub-10ms live API explainability ($\epsilon_i = \frac{(x_i - \hat{x}_i)^2}{\text{MSE}}$). Reserve full SHAP/LIME for asynchronous historical audit tabs.
3. False Positives on Storms	Severe weather causes extreme drops in T and spikes in H , which simple rules flag as broken sensors.	Train a Multivariate LSTM Autoencoder on clean weather physics. Storms preserve physical correlation ($T \downarrow, P \downarrow, H \uparrow$), while sensor faults break physics ($T \downarrow, P$ unchanged, H unchanged).
4. Invisible Calibration Drift	Slow degradation ($\pm 0.05^\circ\text{C/day}$) stays inside valid range bounds, bypassing point-anomaly checks.	Run a 24-Hour Rolling Spatial Buddy Residual Tracker . If Station A's rolling mean deviates steadily from nearby stations over 7+ days, flag Calibration Drift / Maintenance Required .
5. Imputation Liability	Over-writing raw weather data creates legal/meteorological liabilities for national agencies.	Retain and log raw data untouched. Provide estimated values as a separate imputed_value overlay channel with an accompanying confidence score.

2. Ground Truth & Synthetic Anomaly Injection Engine

Ground Truth Selection

- **Source:** Meteostat API (`meteostat` Python package).
- **Time Range:** 2015–2025 (Hourly resolution).
- **Variables:** `temp` ($^{\circ}\text{C}$), `pres` (hPa), `rhum` (%).



```
| to Backend API) |
+-----+
```

Mathematical Fault Injection Models

The AWS Simulator reads clean Meteostat data line-by-line and applies synthetic fault profiles:

```
# 1. Spike / Outlier (Hardware short circuit)
def inject_spike(x_t, std_dev):
    return x_t + np.random.choice([-1, 1]) * np.random.uniform(4.0, 8.0)

# 2. Frozen Value / Sensor Stuck
def inject_freeze(series, start_idx, duration=12):
    series[start_idx : start_idx + duration] = series[start_idx]
    return series

# 3. Calibration Drift (Linear sensor degradation)
def inject_drift(series, start_idx, duration=48, slope=0.1):
    for i in range(duration):
        series[start_idx + i] += slope * i
    return series

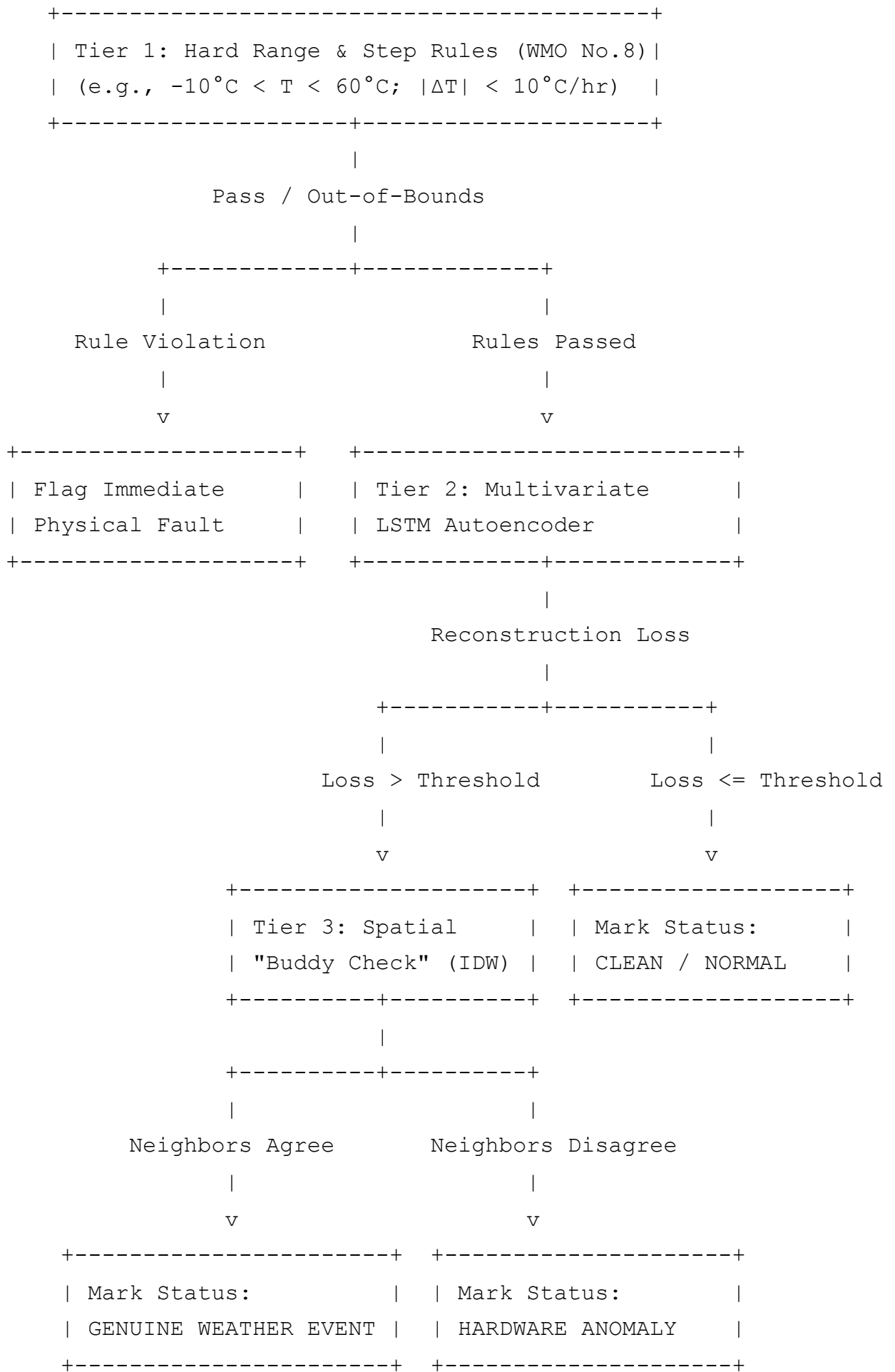
# 4. Communication Failure
def inject_comm_error():
    return None # Missing packet / NaN

# 5. Genuine Weather Event (Extreme storm simulation - PHYSICAL CORRELATION)
def inject_storm(T, P, H):
    T_storm = T - np.random.uniform(8.0, 15.0) # Temperature drops
    P_storm = P - np.random.uniform(10.0, 25.0) # Pressure drops dramatically
    H_storm = np.minimum(100.0, H + np.random.uniform(30.0, 50.0)) # Humidity increases
    return T_storm, P_storm, H_storm
```

3. Hybrid Detection Engine Architecture

The backend implements a **3-Tier Sequential Detection Pipeline**:

```
Incoming Telemetry Payload
{T, P, H, Station_ID, Time}
|
v
```



Model Specification: Multivariate LSTM Autoencoder

- **Input Vector:** $X_t = [T_t, P_t, H_t]$ over a sliding window sequence length of $N = 24$ hours.
- **Architecture:**
- Encoder: `LSTM(input_size=3, hidden_size=64) → LSTM(64, 32)`
- Latent Representation: `Vector(32)`
- Decoder: `LSTM(32, 64) → Linear(64, 3 Guilds / 3 Features)`
- **Loss Function:** Mean Squared Error (MSE) over scaled features (0-1 MinMax Scaling).

$$\text{MSE} = \frac{1}{3} \sum_{i \in \{T, P, H\}} (x_i - \hat{x}_i)^2$$

- **Imputation Engine:** Reconstructed outputs $\hat{X}_t = [\hat{T}_t, \hat{P}_t, \hat{H}_t]$ serve directly as the **Corrected Data Estimates**.

4. Root-Cause, Explainability & Sensor Health Logic

Root-Cause Classification Rules

Once Tier 2/3 flags an anomaly, the Root-Cause Engine categorizes the failure:

```
def classify_fault(raw_window, reconstructed_window, loss_vector, spatial_diff)
    # loss_vector = [mse_T, mse_P, mse_H]
    dominant_channel = np.argmax(loss_vector)  # 0: Temp, 1: Pres, 2: Humidity

    # Check for Frozen Value
    if np.std(raw_window[-6:, dominant_channel]) == 0:
        return "SENSOR_FREEZE", "High"

    # Check for Spike
    if loss_vector[dominant_channel] > 5 * np.mean(loss_vector):
        return "HARDWARE_SPIKE", "High"

    # Check for Spatial Drift
    if spatial_diff > threshold and loss_vector[dominant_channel] < medium_thresh:
        return "CALIBRATION_DRIFT", "Medium"

    # Check for Physical Misalignment
    if loss_vector[0] > thresh and loss_vector[1] < low_thresh and loss_vector[2] < low_thresh:
        return "SINGLE_SENSOR_PHYSICS_BREACH", "High"
```

```
return "UNKNOWN_ANOMALY", "Low"
```

Instant Explainability Vector (Real-Time API Output)

Instead of slow SHAP computation, the API computes the **Reconstruction Error Contribution Percentage**:

$$\text{Contribution}_i = \frac{(x_i - \hat{x}_i)^2}{\sum_j (x_j - \hat{x}_j)^2} \times 100$$

- Example Output: "Temperature contributed 91.2% to the physical anomaly score. Expected 28.4°C based on Pressure (1008 hPa) and Humidity (78%), but received 48.1°C."

Sensor Health & Predictive Maintenance Score

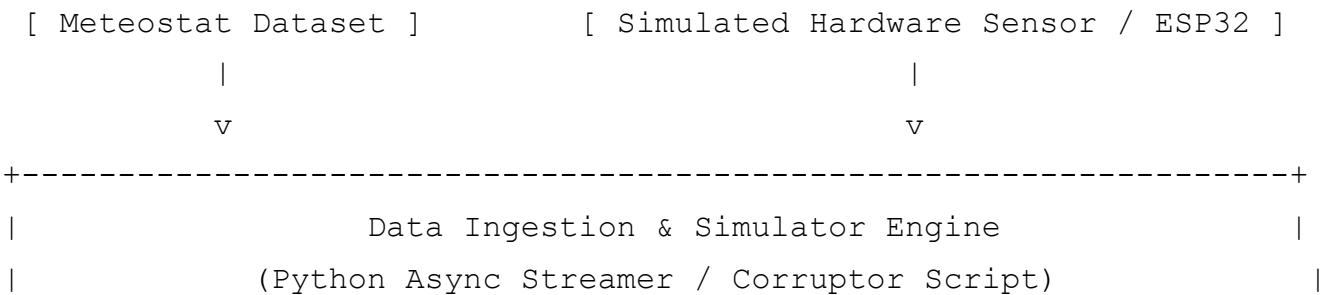
Each station maintains an overall Health Index $H_{station} \in [0, 100]$ computed on a 7-day rolling window:

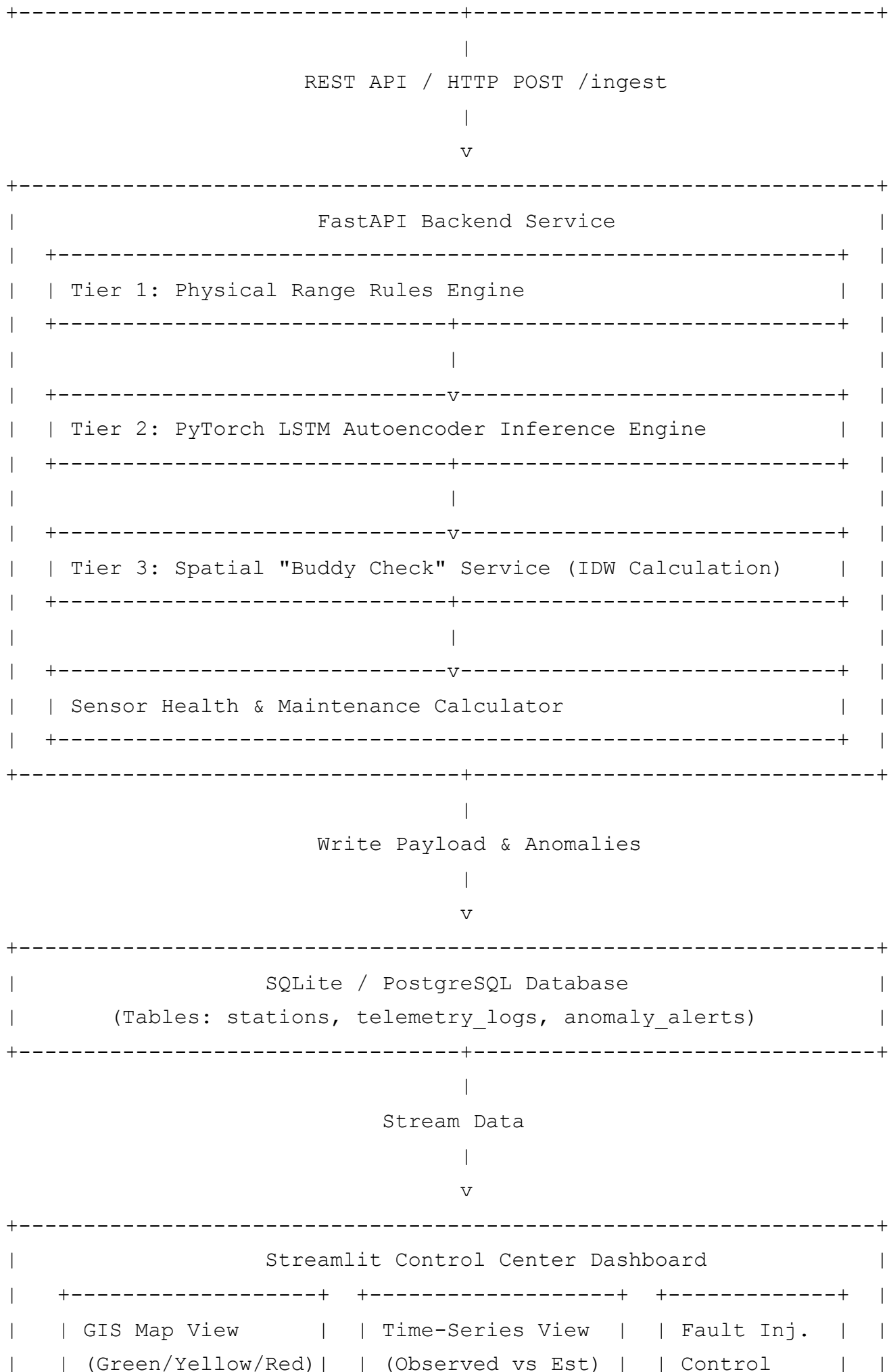
$$H_{station} = 100 - \left(w_1 \cdot F_{spike} + w_2 \cdot F_{freeze} + w_3 \cdot D_{drift} + w_4 \cdot M_{missing} \right)$$

- F_{spike}, F_{freeze} : Count of point faults in last 7 days ($w_1=2, w_2=3$).
- D_{drift} : Cumulative deviation magnitude from spatial neighbors ($w_3=5$).
- $M_{missing}$: Percentage of unreceived packets ($w_4=10$).
- Maintenance Triggers:**
 - $H > 80$: **Healthy (Green)**
 - $50 \leq H \leq 80$: **Degraded / Schedule Calibration (Yellow)**
 - $H < 50$: **Critical Sensor Failure / Dispatch Technician (Red)**

5. System Architecture & Technology Stack

SYSTEM ARCHITECTURE DIAGRAM





Production Tech Stack (100% Free & Open-Source)

- **Language:** Python 3.10+
 - **ML Framework:** PyTorch / PyTorch Lightning
 - **Data Ingestion/Processing:** Meteostat, Pandas, NumPy, Scikit-learn
 - **Backend API:** FastAPI + Uvicorn (Asynchronous, OpenAPI docs)
 - **Database:** SQLite (Zero-config for hackathon) / PostgreSQL
 - **Frontend Dashboard:** Streamlit + Plotly (Interactive time-series & map)
 - **Edge Prototyping:** C++ / Arduino Framework on ESP32 Microcontroller (Optional live hardware hardware demo)
-

6. Database Schema

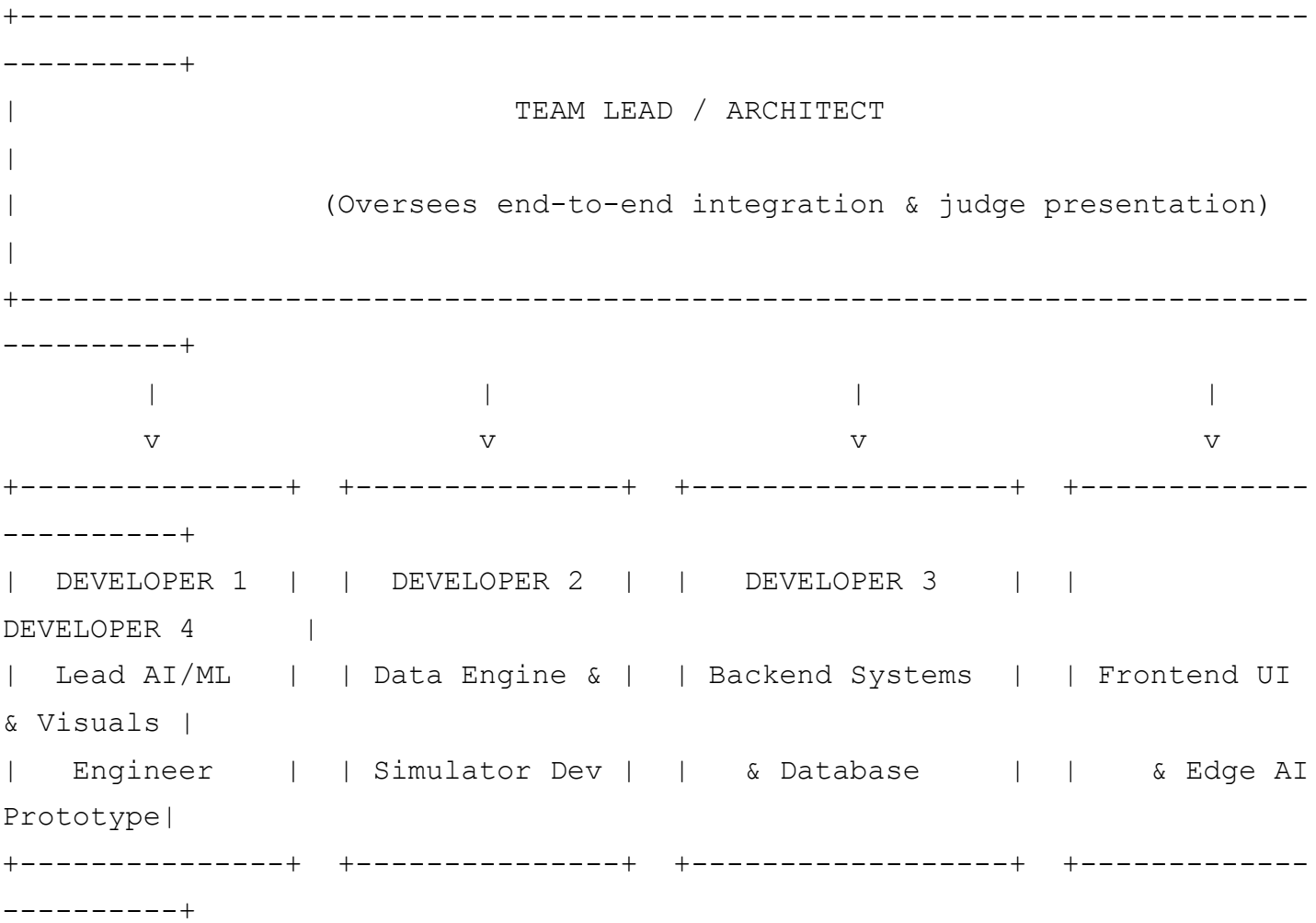
```
-- Stations Metadata Table
CREATE TABLE stations (
    station_id VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100),
    latitude REAL,
    longitude REAL,
    elevation REAL,
    health_score REAL DEFAULT 100.0,
    status VARCHAR(20) DEFAULT 'HEALTHY'
);

-- Ingested Telemetry Logs Table
CREATE TABLE telemetry_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    station_id VARCHAR(20),
    timestamp DATETIME,
    temp_observed REAL,
    pres_observed REAL,
    rhum_observed REAL,
    temp_imputed REAL,
    pres_imputed REAL,
    rhum_imputed REAL,
    is_anomaly BOOLEAN,
    FOREIGN KEY(station_id) REFERENCES stations(station_id)
);
```



```
-- Anomaly Alerts & Root Cause Logs Table
CREATE TABLE anomaly_alerts (
    alert_id INTEGER PRIMARY KEY AUTOINCREMENT,
    station_id VARCHAR(20),
    timestamp DATETIME,
    fault_type VARCHAR(50),          -- SPIKE, FREEZE, DRIFT, COMM_ERROR, ST
    confidence_score REAL,          -- 0.00 to 1.00
    severity VARCHAR(20),          -- LOW, MEDIUM, HIGH, CRITICAL
    explainability_text TEXT,      -- Human readable explanation
    FOREIGN KEY(station_id) REFERENCES stations(station_id)
);
```

7. 5-Person Team Role Distribution



Detailed Task Responsibilities

Developer 1: Lead AI/ML Engineer

- Build and train the PyTorch Multivariate LSTM Autoencoder on clean Meteostat data.
- Implement feature scaling and windowing routines (24-hour sequence sequences).
- Compute reconstruction threshold bounds and design the anomaly classification rules.
- Output serialized model weights (`.pt` / `.onnx`) and build a standalone `predict()` function.

Developer 2: Data Pipeline & Simulator Engineer

- Write the Meteostat automated fetcher and filter out invalid Indian weather stations.
- Build the **Synthetic Anomaly Generator** (Spikes, Freezes, Calibration Drifts, Storms).
- Build the **AWS Hardware Simulator script** that continuously fires real-time HTTP POST JSON requests into the FastAPI backend.
- Prepare labeled test datasets with known synthetic anomalies for final model evaluation metrics.

Developer 3: Backend Systems & Database Engineer

- Set up the FastAPI backend application structure and define database schemas in SQLite.
- Write the 3-Tier Detection Engine logic inside FastAPI (`/ingest` endpoint).
- Build spatial "Buddy Check" routines using Inverse Distance Weighting (IDW).
- Implement the Sensor Health Score calculation algorithm and trigger automated alerts.

Developer 4: Frontend UI & Visualization Lead

- Build the Streamlit dashboard layout (GIS Map, Live Alert Feed, Metrics Cards).
- Construct Plotly interactive time-series graphs with red anomaly markers overlaying clean vs observed data.
- Build the **Interactive Fault Injection Panel** on the UI so judges can trigger live sensor failures.
- Integrate SHAP summary plots and root-cause explanation cards into the UI.

Developer 5: Spatial Logic, Edge AI & QA Lead

- Develop a micro C++ / Arduino sketch for ESP32 running lightweight rule checks to prove Edge AI feasibility.
- Run end-to-end integration tests across Data Generator $\rightarrow$ FastAPI $\rightarrow$ Database $\rightarrow$ Streamlit.
- Calculate model evaluation metrics (Precision, Recall, F1-Score, Detection Latency).
- Prepare presentation slides, architecture diagrams, and choreograph the 3-minute live judge pitch.

8. 48-Hour Tactical Action Plan

Hours 00–12: Foundation & Data Pipelines

- **Dev 2:** Query Meteostat, select 5 Indian stations, extract clean 2015–2025 dataset, build synthetic injection functions.
- **Dev 1:** Build PyTorch LSTM Autoencoder architecture and begin baseline training on clean weather data.
- **Dev 3:** Initialize FastAPI app, set up SQLite database schema, create baseline REST API endpoints.
- **Dev 4:** Design Streamlit wireframe, set up Mapbox/Plotly map components.
- **Dev 5:** Set up Git repository, write ESP32 Edge C++ firmware code structure.

Hours 12–24: Core Intelligence & Engine Integration

- **Dev 1:** Finalize LSTM Autoencoder training, establish optimal reconstruction loss threshold, serialize model.
- **Dev 2:** Finalize AWS Hardware Simulator script streaming live data over async HTTP requests.
- **Dev 3:** Integrate trained PyTorch model into FastAPI inference pipeline (`/ingest` endpoint).
- **Dev 4:** Connect Streamlit frontend to read directly from SQLite DB and update live charts dynamically.
- **Dev 5:** Implement Spatial "Buddy Check" distance calculations across the 5 selected stations.

Hours 24–36: Advanced Features & Simulation Controls

- **Dev 1 & 3:** Implement Root-Cause Classifier, Reconstruction Loss Contribution Vector, and Sensor Health Score algorithm.
- **Dev 2:** Create labeled ground-truth evaluation script comparing model predictions against injected labels.
- **Dev 4:** Build the **Fault Injection Control Panel** slider on Streamlit for live demonstration.
- **Dev 5:** Flash ESP32 board with threshold firmware, verifying local alerting capability.

Hours 36–48: Testing, Polish & Pitch Practice

- **Dev 1 & 2:** Generate final metrics evaluation report (Precision, Recall, F1, Latency).
- **Dev 3 & 4:** UI/UX styling pass, ensuring sub-100ms dashboard updates and crisp Plotly visuals.
- **Dev 5 & Team:** Conduct 5 full dry-runs of the deterministic demonstration and polish presentation slide deck.

9. Model Evaluation Plan & Metrics

You will evaluate your engine against a test set containing **10,000 hourly samples** with a **15% injected anomaly rate**:

Metric	Target Goal	What It Proves to Judges
Precision	$\geq 92\%$	Extremely low false alarm rate; meteorologists won't experience alert fatigue.
Recall (Sensitivity)	$\geq 95\%$	The system catches almost every true hardware failure.
Macro F1-Score	$\geq 93\%$	Balanced accuracy across all fault types (Spikes, Freezes, Drifts).
False Positive Rate (FPR)	$\leq 3\%$	Severe weather events are correctly identified and not flagged as faults.
Inference Latency	$< 15\text{ms}$	High-throughput capability; single server can handle thousands of AWS nodes.

10. Deterministic Demo Script for SIH Judges

Structure the 3-minute judge pitch to demonstrate capabilities sequentially:

Scene 1: Normal Operation (30 Seconds)

- Show the Streamlit GIS map showing 5 Indian stations marked **Green (100% Health)**.
- Point to the live time-series graph displaying normal diurnal temperature/humidity curves. Point out: *"The LSTM Autoencoder is running in real-time with reconstruction loss near zero."*

Scene 2: The Physical Storm Test (60 Seconds) - The Key Differentiator

- Click **"Inject Flash Monsoon Storm"** on the UI.
- Temperature drops 12°C in 30 mins, humidity jumps to 98%, pressure drops 15 hPa .
- Show that **Tier 1 (Rules) passes, Tier 2 (LSTM) reconstructs correctly** because the multi-variable physics hold up, and **Tier 3 (Buddy Check) confirms neighboring stations see the same storm**.
- **System Status:** GENUINE WEATHER EVENT DETECTED (Alert Suppressed) .
Emphasize: *"Traditional rule systems fail here. Our system understands atmospheric physics."*

Scene 3: The Hardware Sensor Failure (60 Seconds)

- Click **"Inject Temperature Sensor Short-Circuit"**.
- Temperature jumps to 65°C while Pressure and Humidity stay completely static.
- System instantly triggers a red alert:
- **Alert:** HARDWARE ANOMALY DETECTED
- **Root Cause:** TEMPERATURE_SENSOR_SPIKE (Confidence: 98.4%)

- **Explainability:** Temperature contributed 94.1% to reconstruction error.
- **Self-Healing Action:** System plots **Corrected/Imputed Temperature (28.2°C)** in dotted blue alongside the corrupted red line.

Scene 4: Predictive Maintenance & Sensor Degradation (30 Seconds)

- Show the **Sensor Health Dashboard**.
- Point out Station 104 whose health score dropped from \$100 \rightarrow 62\$ over a simulated 7-day period due to slow calibration drift.
- **Output:** SCHEDULE MAINTENANCE: Sensor calibration drift detected (+0.08°C/day deviation from nearby stations).

11. SIH Criteria Scoring Matrix

Criteria & Weight	What Judges Look For	How This Blueprint Guarantees Max Points
Innovation & Novelty (25%)	Beyond standard thresholding; novel approach.	Physics-Aware Multivariate Autoencoders combined with Spatial Buddy Checks. Differentiates real storms from hardware faults.
Detection Accuracy (20%)	High F1-score, low false positives.	Rigorous evaluation metrics ($\geq 93\%$ F1) backed by synthetic testing.
Real-Time Capability (15%)	Fast pipeline processing.	Async FastAPI backend delivering sub-15ms model inference.
Explainability (10%)	Transparent decision-making.	Feature-level error contribution vectors + SHAP diagnostic cards.
Scalability (10%)	Handles network expansion.	Decoupled architecture (FastAPI + SQLite/Postgres) scalable to thousands of stations.
Deployability & Edge (10%)	Real-world implementation feasibility.	Low resource consumption (<5MB model footprint) + ESP32 C++ prototype demo.
Visualization / UI (5%)	Clear, intuitive control room interface.	Interactive Streamlit dashboard with GIS maps, Plotly charts, and manual fault injection controls.
Energy Efficiency (5%)	Lightweight execution.	Highly optimized CPU inference without needing continuous GPU compute.

12. Complete Executive Deliverable Summary

Share this checklist with the team to ensure alignment across deliverables:

- **What We Are Building:** A 3-Tier Intelligent AWS Quality Control platform combining physical range rules, multivariate LSTM deep learning, spatial neighborhood verification, instant root-cause explanations, automated data imputation, and predictive sensor health analytics.
- **What We Are NOT Building:** Complex edge-trained neural networks directly on the AWS microcontroller (we use edge for micro-rules, central for ML), or destructive raw data overwrite systems.
- **Data Sources:** Meteostat API historical records (2015–2025) across filtered Indian stations + custom synthetic anomaly injection script.
- **Tech Stack:** Python, PyTorch, FastAPI, SQLite, Streamlit, Plotly, ESP32 (C++).
- **Core Output Features:** Real-time alert feed, confidence/severity scoring, error contribution explainability, corrected value imputation, 7-day rolling sensor health index, interactive fault injection control panel.